



MEAN Training Course

Logical Task

5 Days

Phase 1

Basics

1. HTML
2. CSS
3. Bootstrap
4. JS
5. JQuery
6. Ajax

Phase 2

Basics

1. Node js
2. Mongodb

Udemy videos

1. Node js
2. Mongodb

Phase 3

Basics

1. Angular

Learn google api

3 different tasks

Setup for coding task

1. Gitlab
2. SonarQube
3. OOPS Concept
4. Angular and React template setup

Coding Tasks

9 different tasks

Phase 4

Coding Tasks

12 different tasks

Phase 5

Corporate Level Concepts

1. Hosting/ Domain/ SSL
2. Auto DevOps Pipeline
3. Microservices , Docker And Kubernetes
4. Mongodb Indexing & Mongodb Atlas
5. Redis Management
6. Clustering
7. Security Aspect
8. Load Testing

Phase 6

Product Learning

1. Eber
2. ESuper

Customization on product

1. Eber
2. ESuper

Phase 1

17 Days

- Html, CSS, Bootstrap : **6 days**
- Html, CSS, Bootstrap Task : **3 days**
- JavaScript, jQuery, Ajax : **8 days**

HTML , CSS , Bootstrap

HTML

<https://www.w3schools.com/html/default.asp>

Please at least refer HTML Tutorial and HTML Form tabs from the above link.

CSS

<https://www.w3schools.com/css/default.asp>

Please at least refer CSS Tutorial and CSS Advance from above link.

SCSS

<https://www.w3schools.com/sass/default.php>

Bootstrap

<https://www.w3schools.com/bootstrap4/default.asp>

Please at least refer Bootstrap Tutorial and Bootstrap Grid from above link.

Bootstrap 5 is latest version you can also refer that to be upto-date with latest version.

HTML, CSS, Bootstrap Task



Alex Smith
Founder of Groupeq
There are many variations of passages of Lorem Ipsum available, but the majority have suffered alteration in some form Ipsum available.

142 Projects

61 Comments

154 Tags

22 Followers

54 Articles

32 Friends

Sales in last 24h

206 480



About Alex Smith
There are many variations of passages of Lorem Ipsum available, but the majority have suffered alteration in some form, by injected humour, or randomised words which don't.
If you are going to use a passage of Lorem Ipsum, you need to be sure there isn't anything embarrassing
● Online status

Followers and friends
If you are going to use a passage of Lorem Ipsum, you need to be sure there isn't anything embarrassing




Andrew Williams
Today 4:21 pm - 12.06.2014

Many desktop publishing packages and web page editors now use Lorem Ipsum as their default model text, and a search for 'lorem ipsum' will uncover many web sites still in their infancy. Packages and web page editors now use Lorem Ipsum as their default model text.

👍 Like this! 💬 Comment ➦ Share



Andrew Williams Internet tend to repeat predefined chunks as necessary, making this the first true generator on the Internet. It uses a dictionary of over 200 Latin words.
👍 26 Like this! - 12.06.2014



Andrew Williams Making this the first true generator on the Internet. It uses a dictionary of.
👍 11 Like this! - 10.07.2014



Write comment...



Meeting
Conference on the sales results for the previous year. Monica please examine sales trends in marketing and products. Below please find the current status of the sale.

Today
Dec 24

More info



Send documents to Mike
Lorem Ipsum is simply dummy text of the printing and typesetting industry. Lorem Ipsum has been the industry's standard dummy text ever since.

Today
Dec 24

Download document

Phase 1

HTML, CSS, Bootstrap Task (continue)

- First of all make a design which is shown in above slide using only html and CSS (don't use bootstrap)
- After that make another design using html , CSS and bootstrap
- Please make it responsive in both the cases .

Javascript , JQuery , Ajax

Javascript

✓ https://www.tutorialspoint.com/javascript/javascript_quick_guide.htm

- Overview ✓
- Syntax ✓
- Variables ✓
- Operators ✓
- Conditions ✓
- Loops ✓
- Functions ✓
- Events ✓
- Page Redirect ✓
- Dialog Box ✓
- Page Printing ✓
- Number Object ✓
- Boolean Object ✓
- String Object ✓
- Arrays Object ✓
- Date Object ✓
- Math Object ✓
- Regular Expressions ✓
- RegExp Object ✓
- Document Object Model or DOM ✓
- Error Handling ✓
- Validations ✓
- Animation ✓
- Es6 ✓

Jquery

<https://www.tutorialspoint.com/jquery/index.htm>

- Overview ✓
- Selectors ✓
- Attributes ✓
- CSS ✓
- DOM ✓
- Events ✓
- AJAX ✓
- UI widget ✓
- Utilities ✓

Ajax

<https://www.tutorialspoint.com/ajax/>

- Overview ✓
- AJAX operation ✓
- XMLHttpRequest ✓

Phase 2

20 - 24 Days

- Node JS (with Udemy videos) : 8 - 10 days
- MongoDB (with Udemy videos) : 8 - 10 days
- EJS Task : 4 days

Node JS (with Udemy videos)

<https://www.guru99.com/node-js-tutorial.html>

Introduction of node js

advantages/disadvantages

modules

Node Package Manager (NPM)

package.json

environment variables

http modules

file system

what is npm

events

callback

callback hell

async await

event loop

event emitter

buffer

stream

multer

file system

Node JS (with Udemy videos)

https://www.tutorialspoint.com/expressjs/expressjs_overview.htm

what is express

middlewares

session

routing

body parser

redis

HTTP Methods

form data

authentication

cookies

REPL : https://www.tutorialspoint.com/nodejs/nodejs_repl_terminal.htm

Promise : <https://www.guru99.com/node-js-promise-generator-event.html>

Features of Node.js : <https://www.guru99.com/node-js-tutorial.html>

<https://www.tutorialspoint.com/nodejs/index.htm>

Scaling Application

Global Objects

Phase 2

Node JS (with Udemy videos)

Simple crud operation api : https://www.tutorialspoint.com/expressjs/expressjs_restful_apis.htm

Email send (gmail) : <https://netcorecloud.com/tutorials/how-to-send-email-with-node-js/>

SMS send <https://www.twilio.com/docs/sms/quickstart/node>

Notification send(GCM, FCM , HCM) :

<https://tudip.com/blog-post/how-to-send-push-notifications-to-android-devices-in-nodejs/>

Socket.io demo : <https://socket.io/get-started/chat>

Generate QR code : <https://www.section.io/engineering-education/how-to-generate-qr-codes-using-nodejs/>

Cors : <https://expressjs.com/en/resources/middleware/cors.html>

MongoDB (with Udemy videos)

1. MongoDB introduction

<https://docs.mongodb.com/manual/introduction>

<https://docs.mongodb.com/manual/tutorial/getting-started/>

<https://docs.mongodb.com/manual/core/databases-and-collections>

<https://docs.mongodb.com/manual/core/document>

<https://docs.mongodb.com/manual/reference/bson-types>

2. Crud in MongoDB

https://www.tutorialspoint.com/mongodb/mongodb_insert_document.htm

https://www.tutorialspoint.com/mongodb/mongodb_query_document.htm

https://www.tutorialspoint.com/mongodb/mongodb_update_document.htm

https://www.tutorialspoint.com/mongodb/mongodb_delete_document.htm

MongoDB (with Udemy videos)

3. Indexes in MongoDB

<https://docs.mongodb.com/manual/indexes>

4. setup mongoose package

<https://www.section.io/engineering-education/nodejs-mongoosejs-mongodb/>

5. Crud in mongoose

[find](#)

[findById](#)

[findOne](#)

[findByIdAndUpdate](#)

[findOneAndUpdate](#)

[findOneAndReplace](#)

[updateMany](#)

[updateOne](#)

[findByIdAndDelete](#)

[findByIdAndRemove](#)

[deleteMany](#)

[deleteOne](#)

MongoDB (with Udemy videos)

6. aggregate (including all stages like lookup , match , group , project)

https://www.tutorialspoint.com/mongodb/mongodb_aggregation.htm

<https://docs.mongodb.com/manual/core/aggregation-pipeline/>

<https://docs.mongodb.com/manual/reference/operator/query/>

<https://docs.mongodb.com/manual/reference/operator/aggregation-pipeline/>

<https://masteringjs.io/tutorials/mongoose/aggregate>

7. populate

<https://mongoosejs.com/docs/populate.html>

8. dump and restore

<https://docs.mongodb.com/manual/tutorial/backup-and-restore-tools>

EJS task(CRUD operation)

- Add the username, userProfile, email, and phone number via a form submission with validation and store all details in db.
- Display the complete list of added users in a table format .
- After add , edit and delete particular record the page won't be refresh and reflect the updated data in table (do this using **Ajax**) .
- The two buttons, namely **Edit** and **Delete**, should be provided to enable users to modify or remove user information
- Add validation rules to restrict users from entering duplicate email or phone number entries.
- Search details :
 - Searching can be done using ID, name, phone number, or email
- Pagination
 - implement pagination logic on the server-side

Phase 3

20 - 25 Days

- Angular (with Udemy videos) : 12 - 15 days
- Google tasks : 3 - 4 days
- Setup For Coding Task : 1 day
- Task No. 1 - 9 : 4 - 5 days

Angular (with Udemy videos)

1. Introduction to Angular, Typescript and About Project Folder Structure

<https://www.c-sharpcorner.com/article/learn-angular-8-step-by-step-in-10-days-day-1/>

2. Components in Angular

<https://angular.io/guide/component-overview>

<https://angular.io/guide/lifecycle-hooks>

<https://www.c-sharpcorner.com/article/learn-angular-8-step-by-step-in-10-days-component-day-2/>

3. Data Binding in Angular

<https://www.c-sharpcorner.com/article/learn-angular-8-step-by-step-in-10-days-data-binding-day-3/>

<https://angular.io/guide/two-way-binding>

[event-binding](#) [property-binding](#) [attribute-binding](#)

<https://angular.io/guide/template-reference-variables>

Angular (with Udemy videos)

4. Directives in Angular

<https://www.c-sharpcorner.com/article/learn-angular-8-step-by-step-in-10-days-directives-day-4/>

<https://angular.io/guide/built-in-directives>

<https://angular.io/guide/attribute-directives>

<https://angular.io/guide/structural-directives>

5. pipes

<https://angular.io/guide/pipes>

<https://www.c-sharpcorner.com/article/learn-angular-8-step-by-step-in-10-days-pipes-day-5/>

Angular (with Udemy videos)

6. routing and navigations

<https://angular.io/guide/routing-overview>

<https://angular.io/guide/router>

<https://angular.io/guide/router-tutorial>

<https://angular.io/guide/routing-with-urlmatcher>

<https://angular.io/guide/router-tutorial-toh>

<https://angular.io/guide/router-reference>

<https://www.c-sharpcorner.com/learn/angular-8-in-10-days/angular-8-route-or-component-navigation-day-10>

Angular (with Udemy videos)

7. forms

<https://www.c-sharpcorner.com/article/learn-angular-8-step-by-step-in-10-days-angular-forms-day-7/>

<https://angular.io/guide/forms-overview>

<https://angular.io/guide/reactive-forms>

<https://angular.io/guide/form-validation>

<https://angular.io/guide/dynamic-form>

9. dependency injections

<https://angular.io/guide/dependency-injection>

<https://angular.io/guide/dependency-injection-providers>

9. services

<https://www.c-sharpcorner.com/article/angular-8-service-day-8/>

Angular (with Udemy videos)

10. view encapsulations

<https://www.c-sharpcorner.com/article/learn-angular-8-step-by-step-in-10-days-view-encapsulation-day-6/>

<https://angular.io/guide/view-encapsulation>

<https://angular.io/api/core/ViewChildren>

<https://angular.io/api/core/ViewChild>

11. HttpClient (Api Calling)

<https://www.c-sharpcorner.com/article/learn-angular-8-step-by-step-in-10-days-httpclient-or-ajax-call-day-9/>

<https://angular.io/guide/http>

12. interceptor

<https://angular.io/api/common/http/HttpInterceptor>

Angular (with Udemy videos)

13. eventemitter and observable

<https://angular.io/api/core/EventEmitter>

<https://angular.io/guide/observables>

<https://angular.io/guide/rx-library>

<https://angular.io/guide/observables-in-angular>

14. animations

<https://angular.io/guide/animations>

<https://angular.io/guide/transition-and-triggers>

<https://angular.io/guide/complex-animation-sequences>

<https://angular.io/guide/reusable-animations>

<https://angular.io/guide/route-animations>

Angular JS (with Udemy videos) (12 - 15 Days)

15. extra

<https://angular.io/guide/lazy-loading-ngmodules>

<https://angular.io/guide/web-worker>

<https://angular.io/guide/service-worker-intro>

<https://angular.io/guide/service-worker-notifications>

Subject and Observable

Google API

<https://developers.google.com/maps/documentation/javascript>

1. Create google api console account (To create an account, you must specify the United States as your country during the sign-up process)
2. Add billing account, you can add your card, it will give monthly \$300 credit free for your development testing (The Rupay card is not accepted, but Visa and Mastercard are both accepted forms of payment)
3. You should know how to enable api
4. Perform next 3 tasks
5. You should know which api is used for which task. For example
 - a. which api is used for autosuggestion
 - b. Which api is used to calculated distance and time
 - c. Which api is used to draw path
 - d. How much each charge
 - e. What is daily quota
 - f. So that you know you need to code such a way that you do optimized utilization of api

Google API Task 1 - LOCATION SEARCH

1. Show current location in map on page load
2. Show suggestion for places using google API when user type in textbox
3. Show selected location in map through pin
4. User can move pin in map, change address in map accordingly

Google API Task 2 - ESTIMATED TIME & DISTANCE

1. Enter from location using google suggestion
2. Enter to location using google suggestion
3. Show both source and destination pin in map when selected from suggestion
4. Once both selected, show estimated distance and time between both to travel
5. Draw path between pickup and drop off based on Google's suggested shorted path

Google API Task 3 - ZONE

1. Draw polygon (we call it zone in our Eber product) using drawing library in google map
2. Enter one location from textbox
3. Result should return if that selected location falls under drawn polygon
4. If it belong, show green message (Yes, Your entered location belongs to drawn zone.)
5. If it doesn't belong, show red message (Sorry! Entered location doesn't belong to drawn zone.)

Setup For Coding Task

Gitlab

1. Create GitLab account (its free) <https://gitlab.com>
2. Daily you should commit your work done on GitLab account

SonarQube

1. Integrate SonarQube with your system to check code quality. <https://www.sonarqube.org/>
2. If your code score is not above 95% you should not commit to GitLab and consider your task incomplete

OOPS Concept & Structure

1. You might have installed many modules till now. Check out code of those modules with your code
2. Module code will be in proper structure following OOP concepts
3. You won't see any hanging functions in the air. Everything will be accessible through class object only

Angular Template Setup

1. Try to find one nice free angular admin panel template and setup in your system
2. All tasks mentioned in this doc should be created as one tab in that panel.
3. All tabs are related to each other so don't skip any 😊
4. Perform all task using MEAN stack (view should be in angular js)

Task 1 - LOGIN/LOGOUT

1. Create admin user
2. When he login using his id and password then only, he should be able to access , other functionality we will be doing next
3. When he logout his session will be over
4. Manage session using login/logout in all pages
5. If admin is ideal for more than 20 minutes then he should be automatically logged out

Task 2 - MENU

Create below structure in your menu (side bar)

- ❑ Rides
 - Create Ride
 - Confirmed Rides
 - Ride History
- ❑ Users
- ❑ Drivers
 - List
 - Running request
- ❑ Pricing
 - Country
 - City
 - Vehicle type
 - Vehicle Pricing
- ❑ Settings

Task 3 - Vehicle Type

1. Create new form to add vehicle type like sedan, SUV etc.
2. Enter name and icon of vehicle you want to add. Icon should have size validation and Name shouldn't be empty
3. Once a vehicle type is added, the vehicle type list should be displayed in a grid view below the form.
4. You should be able to edit added vehicle details.

Task 4 - COUNTRY

1. Use Rest countries api to add country <https://restcountries.com>
2. To add country open popup(modal) when user click on add country button
3. Once you input the name of the country, other details such as the currency , country code, and country calling code will be automatically filled
4. Please add a validation to ensure that duplicate entries for countries are not allowed.
5. Editing and deleting countries is not permitted.
6. With country you should store associated details like time zone, currency, country code, flag symbol etc.
7. After adding a new country, the list of countries should be displayed in a grid view.
8. User should be able to search from added country based on name filter

Task 5 - CITY

- **CREATE ZONE INSIDE COUNTRY**

1. Add a dropdown menu that contains only the countries that are stored in the database
2. Now that you learned how to create zone so create zone inside country
3. This zone will act as my city or area of that country
4. Implement Google Autocomplete for the input field where users can enter the city name
5. When selecting a country, the autocomplete feature should display suggestions based on the chosen country
6. Include a validation to prevent the entry of duplicate city names
7. Ensure that the zone field is editable, while the country field remains uneditable.
8. When selecting a country from the dropdown menu, any previously added cities (zones) that belong to that specific country should be displayed.

Task 6 - VEHICLE PRICING

1. Now that you have vehicle type and country/zone (city) then lets associate them
2. Per city you can add different vehicle type so select country from dropdown from previously added list
3. Once you select country, in next dropdown it should show me list of area (zones) created in that country and I should be able to select one
4. For each vehicle type which you created for each country you should be able to set pricing
5. There can be multiple type of pricing (i.e. At a time, one or more types of pricing may be used)
 1. Time price
 2. Distance price
 3. Min price
 4. Driver profit

Country	India ▼
City	Bengaluru ▼
Type	SUV ▼
Driver Profit	?
Min. Fare	?
Distance for Base Price	0 ▼ ?
Base Price	?
Price Per Unit Distance	?
Price Per Unit Time (min)	?
Max Space	?

Task 7 - Settings

1. Set the seconds for the driver to accept the request by providing a selection drop-down menu with options of 10, 20, 30, 45, 60, 90, and 120 seconds in duration
2. Set how many stop user can add between both location. (1 to 5)

Task 8 - USER

1. Add user details **WITHOUT PAGE REFRESH** in user table of Mongo DB :
User profile , User name, User email, User phone with country code (country selection option).
2. Validation for email, phone number, empty fields and duplication fields
3. Display user details in table once added
4. Delete particular user detail without page refresh
5. Search detail :
Admin can search by Id, name, phone number, email.
6. Sort detail based on selected column
7. Implement searching, sorting, and pagination exclusively on the server-side
8. Admin can edit user information.
9. For a simplicity you can put dropdown menu and give edit and delete button inside it so using it admin can delete or edit user information .
10. Add a **Cards** button to the dropdown menu too. So When the user clicks on it, a modal will open, displaying all cards previously added by the user. The user can add a new card or delete an existing one and set a default card from the list(Stripe implementation).

Task 9 - DRIVER - LIST

1. Add driver details **WITHOUT PAGE REFRESH** in user table of Mongo DB :
Profile , Name, Email, Phone with country code (country selection option), city selection .
2. Validation for email, phone number, empty fields and duplication fields
3. Display driver details in table once added
4. Add toggle button so using that admin can approve or decline driver. (Approve driver can be declined and decline can be approved)
5. Search detail :
Admin can search by Id, name, phone number, email.
6. Sort detail based on selected column
7. Implement searching, sorting, and pagination exclusively on the server-side
8. Admin can edit driver information.
9. Admin can assign service type to driver. Which you created in previous task. So that is link with driver.(To assign service type open popup(modal) when admin click on add service type button)
10. Admin can remove service type after assign or change service type.
11. For a simplicity you can put dropdown menu and give edit ,delete and assign vehicle button inside it so using it admin can delete or edit driver information or assign service.

Phase 4

18 - 22 Days

- Task No. 10 - 18 : 15 - 18 days
- Task No. 19 - 21 : 3 - 4 days

Task 10 - SOCKET.IO

1. Learn what is it
2. All status of request and action (accepted , cancelled , change driver etc.) handle by Socket (For more information about the status, please refer to the next slides.)

HOW PRICING WORKS

- **Minimum Fare :** The minimum fare for an ride is used to ensure that drivers receive a reasonable payment for their time and effort. In some cases, the minimum fare may also be used to discourage riders from taking short trips that are not cost-effective for drivers. This helps to ensure that drivers are compensated fairly for their time and effort and helps to encourage them to accept more rides.
- **Base Price :** The base price is a predetermined amount that a rider is charged as soon as the ride starts. This is a fixed amount that is set by admin and varies by city and location.
- **Base Price Distance:** The base price distance is a predetermined distance that user can travel with base price. For example if admin set base price 2 and base distance 5 km for starting 5 km or less than that, it will cost you \$2.
- **Unit Distance Price:** The unit distance price is the amount that a rider is charged for each mile or kilometer traveled after base price distance.
- **Unit time Price:** The unit time price is the amount that a rider is charged for each minute that they spend in the vehicle during a ride.
- **Driver Profit:** The driver profit is the amount of money that a driver earns from their gross earnings.

HOW PRICING WORKS

predefined values

Driver Profit	80%
Min Fare	25
Base Price	20
Base Price Distance	1
Unit Distance Price	10
Unit Time Price	1

Example-1

Total Distance	1.2
Total Time	2

Base Price	20.00
Distance Price	2.00
Time Price	2.00
Service Fees	25.00

Example-2

Total Distance	5.35
Total Time	12
Base Price	20.00
Distance Price	43.50
Time Price	12.00
Service Fees	75.50

Example-3

Total Distance	0.9
Total Time	6
Base Price	20.00
Distance Price	0.00
Time Price	6.00
Service Fees	26.00

Phase 5

Task 11 - RIDES -> CREATE REQUEST

On this page we will book request with below data.

1. At first, only the phone number field is enabled, while the other fields are disabled. User details are retrieved based on the phone number. If the user does not exist, an error message is displayed. Otherwise, move further.
2. We provide two radio buttons for payment options: cash and card. If the user selects cash, no action is taken. If the user selects card, the payment amount will be deducted when the trip ends.
3. Ride details such as the pickup location and drop-off location can be added. Additionally, users can add multiple stops between the pickup and drop-off locations. The number of stops that can be added is determined by the admin and can be set in the settings.
4. The pickup location, drop-off location, and pins for multiple stops will be displayed on map only after the user has added them
5. Show Estimate time and distance
6. Show service type list from pickup location with estimate fare.
7. The user has the option to book a ride either immediately ('NOW') or for a later time ('SCHEDULE') (show date and time picker for schedule)
8. Book ride button, check all validation and book trip with all details.

Task 12 - RIDES->CONFIRMED RIDES

On this page, we will show all booked requests by the user for the admin

1. In the list , we have to show details such as request id, user name, user id, pick up time, pick up address drop off address, service type, option for assigning driver if not found, and cancel option.
2. Admin can cancel the ride before accepted. Give the Cancel button. If the ride is accepted, then we will show a status of a request.
 - Accepted
 - Arrived
 - Picked
 - Started
 - Completed
3. Click on the request we will show details information on the same page. (all addresses, estimate distance and time, fare, status with time, user details with profile, create time and schedule time if schedule request)
4. We can display an 'Assign' button on the row of the request table. (Follow the instructions provided in the next slide when the admin clicks on the 'assign' button)
5. Admin can filter requests using status, filter by vehicle, search by username, phone number, request-id, and from date .
 - If the driver accepts the request, display the **driver's name** instead of the Assign button. (Use here **socket.io**)

Phase 5

Task 13 - DRIVER -> Running request

We will show all assign request here (Use here **socket.io**)

1. Display all assigned ride requests, and when a request is clicked, show its details and information.
2. From here, the driver (admin) can either accept or reject the ride request
3. We will also display a timer for accepting the ride request.
4. If a driver cancels a request and you select the **Assign Selected Driver** option, the request will be cancelled entirely.
However, if you choose the **Assign Any Available Driver** option, the request will be reassigned to another driver and will not be offered to the same driver again

Task 14 - RIDES -> CONFIRMED RIDES -> Assign Dialog

When we click on the ASSIGN button of the request table we will do the below thing in Dialog.

1. Display trip details on a dialog box, including all locations added by the user, booking time or scheduled time.
Additionally, show a list of available drivers with their ID, name, phone number, and email address.
2. On the dialog box, we will display two buttons: **Assign Selected Driver** and **Assign Any Available Driver**
 1. The admin can select a driver from the list and click on the **Assign Selected Driver** button, which will send the request to the selected driver
 2. When the **Assign Any Available Driver** button is clicked, the request will be sent to the **any available** driver.
3. After the request has been assigned to a driver, there will be a predetermine duration window for the driver to accept the request (the admin can set the duration for the driver to accept in the Settings).
4. The nearest driver who meets the specified criteria will receive the ride request
 1. Driver approved
 2. Driver have same vehicle
 3. Driver do not have any request running

Task 15 - CRONE TO HANDLE TIMEOUT

1. If the driver does not accept or reject the request within the set time, the request will time out. The user will be notified that no driver is available, and the **Reassign** button will be displayed on **CONFIRMED RIDES** Table.(Use here **socket.io**)
2. If the first driver rejects the request or the request times out after being automatically sent to any available driver, the request will automatically send to the any available driver through a cron job. This process will continue until all the any available drivers receive the request.
3. When the user clicks on the Reassign button, the same process will be followed as when assigning a driver.
4. If the driver accept that particular request then so according to that admin can change the status of that request including arrived, started , completed .
5. If the trip is completed and user select the cash as a payment option then move it directly to the confirm ride section and if payment option was card then complete the payment process on that particular page and after redirecting it to the confirm ride section .
6. **Note: For Card payment please refer task No. 19 and implement card payment after complete task No. 19 .**

Task 16 - PUSH NOTIFICATION

1. When driver is not found admin should be notified using browser push with sound
2. Make counter on top whenever such notification come counter should increase
3. When admin take action and driver again found counter should decrease
4. Also learn how to send push notification in android and ios app, Both have different ways of sending it.

Task 17- APIS & POSTMAN

Till now you created request and assigned to driver. Further steps are

1. Driver started moving to user
 2. Driver arrived
 3. Driver started trip
 4. Driver ended trip
 5. Invoice
 1. It will consider distance between start trip and end trip
 2. And apply pricing which we have set in task 9
 6. Feedback
1. Learn what is it
 2. For rest of the task, you need to prepare APIs which will be consumed by app developers
 3. Create API for all mentioned status shown
 4. You can set flag or counter however you prefer to set each status
 5. Taste your API using Postman tool

Task 18 - SHOW RIDE HISTORY

1. You should be able to see all cancelled and completed trips in a single table format
2. Provide various filter options such as trip status, date range, pickup location, drop-off location, and so on. Also, provide a search bar to search for specific trips by keywords. Additionally, allow users to export the trip history in a downloadable format, such as CSV
3. When a ride is clicked, the request details information, map, and travel path should be displayed using **polyline** (google maps).

Task 7 - Settings(Continue)

1. Stripe settings
2. Email settings
3. SMS settings

Task 19 - STRIPE PAYMENT GATEWAY

1. **Create account** in stripe. It is free. Make sure you select country as US as few features are not available in India
2. **Add card** in user database you created using stripe payment gateway
(3D Secure card) : if user enter 3D secure card then do all necessary process and after that add particular card.
3. **Deduct money from card** on end trip using stripe payment gateway from user's added card and should be credited to admin's bank account
If the card is 3D Secure then do all necessary process and complete payment.
4. **Add bank account** in driver using stripe
5. **Transfer money** to driver bank
6. Take stripe key from setting, admin will set there

Task 20 - EMAIL SETUP USING NODEMAILER

1. Learn how to do it
2. For certain event user/driver should be able to receive emails
3. We will send email from server as it is free
4. Few email action
 1. Welcome email on registration
 2. Invoice email on end trip
 3. If you search uber invoice email on internet you will get idea how you should design it
5. Email configuration data set on Setting

Task 21 - SMS SETUP USING Twilio

1. Learn how to do it
2. For certain event user/driver should be able to receive SMS
3. We will send SMS from server
4. Few SMS action
 1. Driver accepted request
 2. Driver started request
 3. Driver completed request
 4. Request payment paid
5. SMS configuration data set on Setting

Phase 5

12 – 15 Days

Corporate Level Concepts

1. Hosting/ Domain/ SSL
2. Auto DevOps Pipeline
3. Microservices , Docker And Kubernetes
4. Mongodb Indexing & Mongodb Atlas
5. Redis Management
6. Clustering
7. Security Aspect
8. Load Testing

Phase 7

17 - 22 Days

- Product learning : 12 - 15 days
- Product customisation: 5 - 7 days

Product learning (Only 1 Product)

Eber – Taxi

1. Admin panel, User website, driver panel, partner panel, corporate panel, dispatcher panel, hotel panel and hub panel.
2. User and Driver application in an Android and iOS
3. It will work on multiple country and city business with N number of service type.
4. User can book request using application and web, same driver will get requests in app side only.

Esuper – All in one

1. Esuper is combination of below business.
 1. Taxi
 2. Courier
 3. Delivery (food , flower , grocery etc.)
 4. Services (car wash, plumber, electrician, painter etc.)
 5. Appointment (Doctor, Hair saloon, Tutor etc.)
 6. E-commerce
2. Admin panel, website + user panel, merchant panel
3. Customer app, Merchant app, Partner app

All the best.